

Лабораторная работа №2

Мониторинг приложения и инфраструктуры с использованием Prometheus

Цель работы

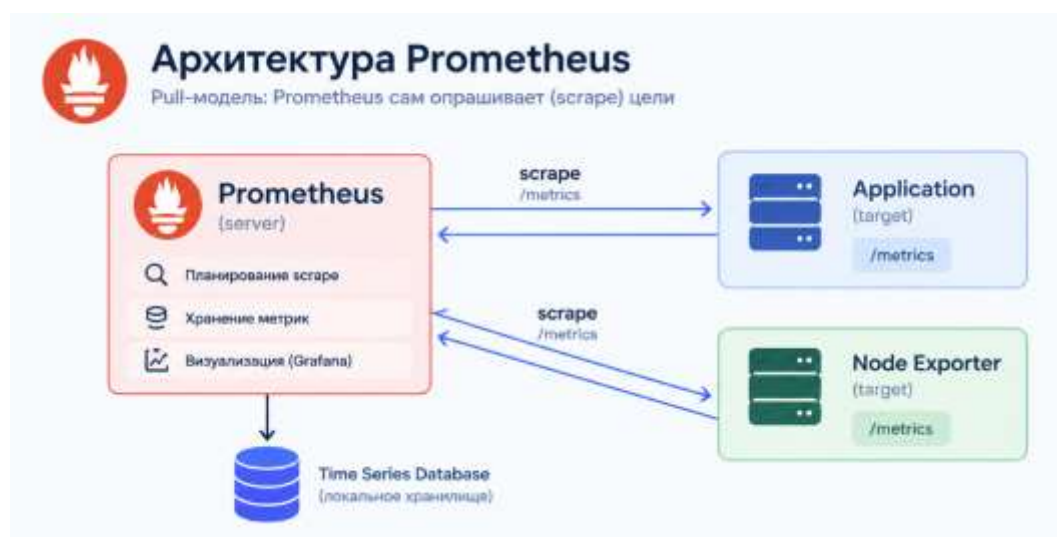
- понимать назначение мониторинга в высоконагруженных системах;
- понимать отличие внешних и внутренних метрик системы;
- познакомиться с архитектурой Prometheus и принципом Pull-модели;
- работать с метриками и временными рядами;
- использовать labels для фильтрации метрик;
- выполнять запросы на языке PromQL;
- отличать counter, gauge, histogram и summary;
- анализировать количество запросов, ошибки, CPU и память;
- сопоставлять метрики Prometheus с результатами нагрузочного тестирования k6;
- находить признаки насыщения ресурсов;
- формулировать гипотезу о причине снижения производительности.

Мониторинг – это процесс постоянного сбора и анализа показателей работы системы. Мониторинг позволяет наблюдать:

- состояние приложения;
- загрузку CPU;
- использование памяти;
- количество запросов и ошибок;
- время обработки запросов;
- состояние Pod'ов;
- изменение показателей во времени.

Prometheus – это система мониторинга и хранения временных рядов и предназначен для:

- сбора метрик;
- хранения метрик;
- выполнения запросов к метрикам;
- анализа изменения показателей во времени;
- обнаружения проблем в работе системы.



Prometheus периодически сам обращается к источникам метрик и забирает их значения. Такой подход называется **Pull-моделью**. Prometheus самостоятельно выполняет запрос GET /metrics и получает текущие значения метрик.

Endpoint /metrics

Многие приложения предоставляют специальный HTTP endpoint:
http://**application**:8080/metrics

Ответ может содержать:

```
http_requests_total 15420
http_errors_total 23
```

Prometheus периодически получает эти значения и сохраняет их вместе со временем измерения.

Метрика – это числовой показатель, описывающий состояние системы.

Например:

```
http_requests_total
http_errors_total
process_cpu_seconds_total
process_resident_memory_bytes
```

Prometheus хранит значения метрик как временные ряды:
metric + labels + timestamp + value

Например:

```
http_requests_total{status="200"} 1000 @ 10:00:00
http_requests_total{status="200"} 1250 @ 10:00:15
http_requests_total{status="200"} 1500 @ 10:00:30
```

Labels позволяют различать разные варианты одной метрики. Это позволяет получать отдельные временные ряды для разных категорий запросов.

Например:

```
http_requests_total{method="GET"}
http_requests_total{method="POST"}

http_requests_total{status="200"}
http_requests_total{status="404"}
http_requests_total{status="500"}
```

Типы метрик

Тип	Характеристика	Пример
Counter	Моноotonно возрастающий счётчик, значение которого увеличивается в течение жизненного цикла процесса и может быть сброшено при его перезапуске.	<code>http_requests_total</code>
Gauge	Изменяемая величина, значение которой может как увеличиваться, так и уменьшаться во времени.	<code>container_memory_working_set_bytes</code>
Histogram	Распределение наблюдаемых значений по заданным интервалам (бакетам) с дополнительным подсчётом общего количества наблюдений и их суммы.	<code>http_request_duration_seconds</code>
Summary	Распределение наблюдаемых значений с вычислением квантилей на стороне приложения (клиента); также хранит количество наблюдений и их сумму.	<code>rpc_duration_seconds</code>

Примечание:

1) для `latency` используют `histogram`, потому что он позволяет считать перцентили через `histogram_quantile()`.

2) `rate()` предназначена для вычисления средней скорости изменения `Counter`. Для `Gauge` обычно анализируют текущее значение или используют функции, подходящие для конкретной задачи

Для работы с метриками используется язык запросов **PromQL**.

Например:

`http_requests_total` – возвращает значения метрики.

`http_requests_total{status="500"}` – фильтрация по label:

Установка Prometheus

Для установки используем Helm – пакетный менеджер для Kubernetes.

Шаг 1. Добавление репозитория

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
helm repo update
```

Шаг 2. Установка

```
helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace
```

Примечание!

`helm install prometheus` – устанавливает Helm-релиз с именем `prometheus`;
`prometheus-community/kube-prometheus-stack` – использует готовый Helm Chart для Prometheus и компонентов мониторинга Kubernetes;
`--namespace monitoring` – устанавливает все компоненты в namespace `monitoring`;
`--create-namespace` – автоматически создаёт namespace `monitoring`, если он ещё не существует.

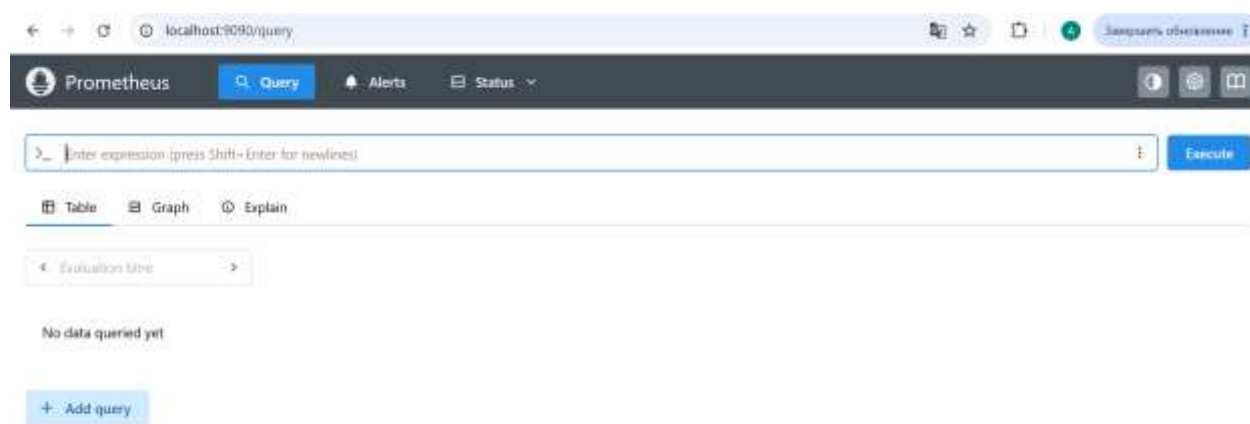
Шаг 3. Проверка установки

```
kubectl get pods -n monitoring
```

Шаг 4. Доступ к Web-интерфейсу

```
kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090
```

Открыть в браузере: `http://localhost:9090`



Пример

Шаг 1. Создадим простое приложение, затем создадим Dockerfile и соберем образ:



```
app.py - Блокнот
Файл Правка Формат Вид Справка
from flask import Flask
from prometheus_client import Counter, Histogram, generate_latest

app = Flask(__name__)
REQUESTS = Counter('http_requests_total', 'Total requests', ['method', 'endpoint', 'status'])
LATENCY = Histogram('http_request_duration_seconds', 'Request latency', ['endpoint'])

@app.route('/')
def index():
    with LATENCY.labels(endpoint='/').time():
        REQUESTS.labels(method='GET', endpoint='/', status='200').inc()
    return "Hello from app!"

@app.route('/metrics')
def metrics():
    return generate_latest(), 200, {'Content-Type': 'text/plain'}
```

Шаг 2. После этого создадим deployment и service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web-app
        image: web-app:latest
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 5000
          name: http

apiVersion: v1
kind: Service
metadata:
  name: web-app-service
  labels:
    app: web-app
spec:
  selector:
    app: web-app
  ports:
  - port: 80
    targetPort: 5000
    name: http
```

Шаг 3. Применим манифесты и проверим приложение

Hello from app!

```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 301.0
python_gc_objects_collected_total{generation="1"} 351.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 77.0
python_gc_collections_total{generation="1"} 6.0
python_gc_collections_total{generation="2"} 0.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="16",version="3.11.16"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 1.93183744e+08
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 3.3869824e+07
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
```

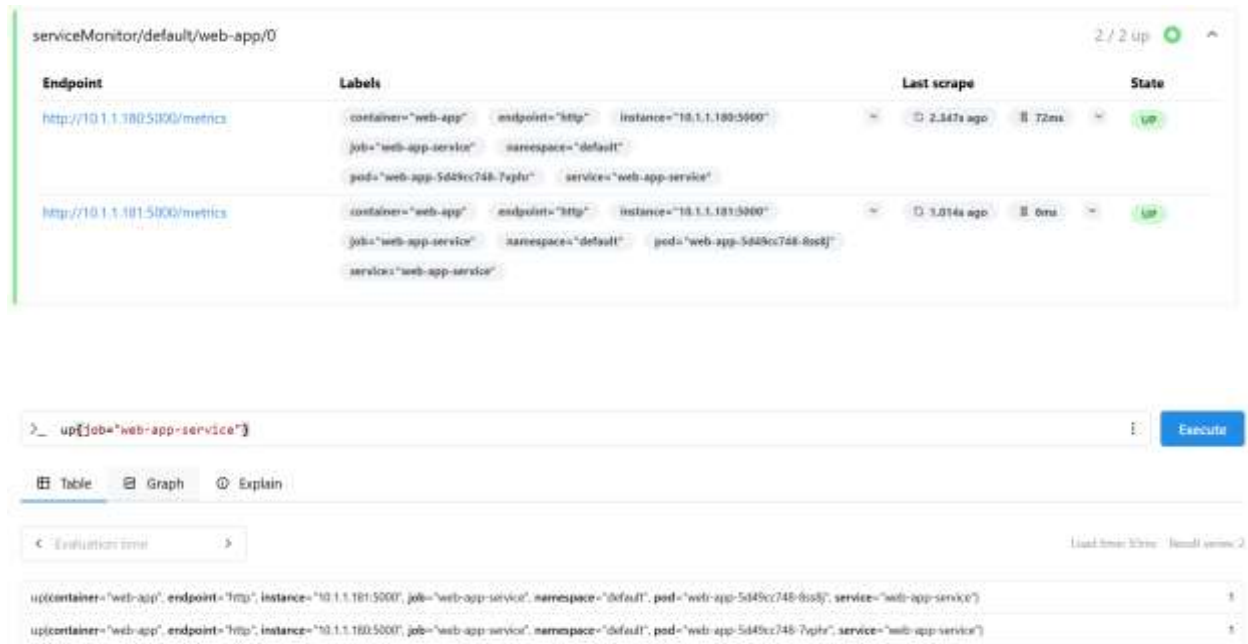
Шаг 4. Создадим ServiceMonitor

ServiceMonitor — это объект Kubernetes (Custom Resource) из экосистемы Prometheus Operator, который говорит Prometheus: «какие Service'ы опрашивать и как».

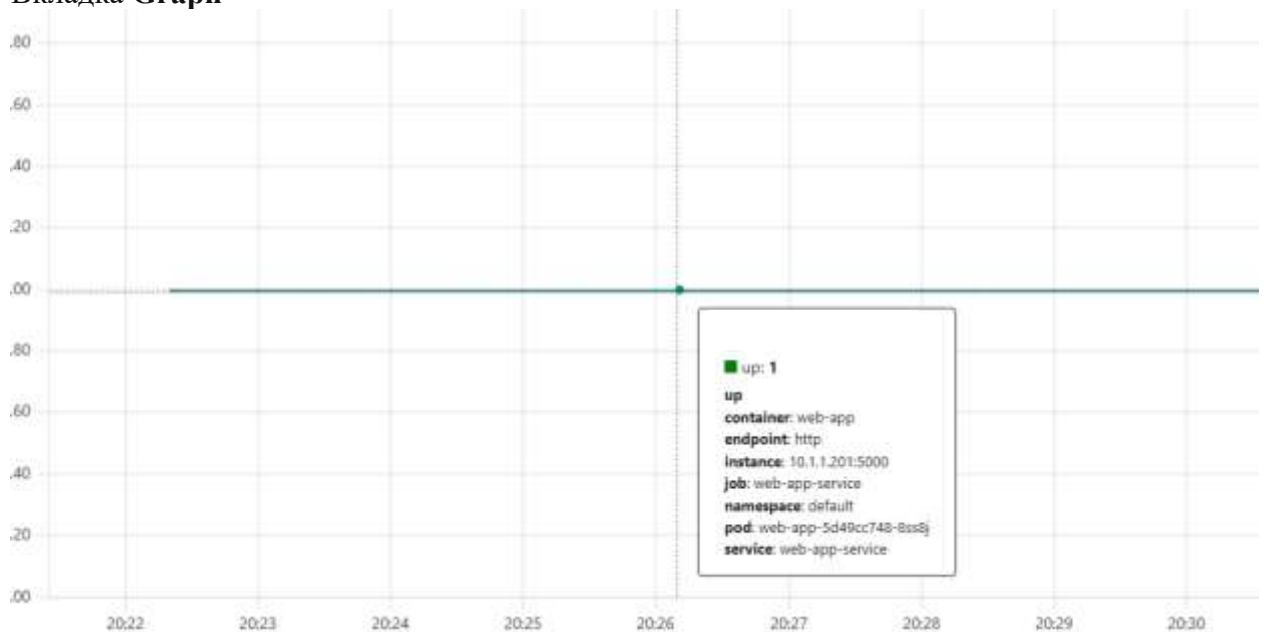
```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: web-app
  labels:
    release: prometheus
spec:
  selector:
    matchLabels:
      app: web-app
  endpoints:
    - port: http
      path: /metrics
      interval: 5s
```

Применим манифест

Шаг 5. Проверим в Prometheus → Status → Targets, что target web-app в состоянии UP.



Вкладка Graph



Шаг 6. Проверить метрики приложения



Шаг 7. Проверить RPS и latency**Шаг 8.** Запустить в другом терминале нагрузочный тест k6

Использовать сценарий из ЛР1 (test-stages.js или test-constant.js).

Во время теста необходимо наблюдать за метрикам. А в Prometheus параллельно смотреть:

```
sum(rate(http_requests_total[1m]))
```

```
sum(rate(http_requests_total{status=~"5.."}[1m]))
```

```
100 * sum(rate(http_requests_total{status=~"5.."}[1m])) /  
sum(rate(http_requests_total[1m]))
```

```
sum(rate(container_cpu_usage_seconds_total{container="web-app"}[1m]))
```

```
sum(container_memory_working_set_bytes{container="web-app"}) / 1024 /  
1024
```

```
histogram_quantile(0.95,  
sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

```
histogram_quantile(0.99,  
sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

Связь k6 и Prometheus

Они позволяют посмотреть по разному на одну систему.

k6: RPS, latency, p95, p99, errors.

Prometheus: CPU, Memory, requests, errors, Pod status.

Важно: RPS и p95 из k6 и из Prometheus могут не совпадать, потому что:

- k6 измеряет latency с клиента (включая сеть), Prometheus – на сервере;
- k6 использует свои бакеты, Prometheus – свои;
- k6 считает за интервал теста, Prometheus – за [1m].

Совместный эксперимент k6 + Prometheus

Одновременно в Prometheus наблюдаем изменение внутренних метрик.

Нагрузка 10 VU, 50 VU, 100 VU, 200 VU

Сопоставление результатов

VU	RPS	p95 (ms)	p99 (ms)	Errors (%)	CPU (%)	Memory (MB)
10						
50						
100						
200						

Главная задача – найти связь между внешними и внутренними показателями.

Например:

$VU \uparrow \rightarrow RPS \uparrow \rightarrow CPU \uparrow \rightarrow p95 \uparrow$

Это может свидетельствовать о приближении системы к ограничению CPU.

Или

$VU \uparrow \rightarrow RPS \text{ почти не меняется} \rightarrow CPU \text{ низкий} \rightarrow Latency \uparrow$

В таком случае причина может быть не в CPU. Узким местом может быть:

- Database
- Network
- External service
- Connection pool
- Lock

Поэтому мониторинг позволяет не просто обнаружить проблему, а сформулировать гипотезу о её причине.

Задания для самостоятельного выполнения**Задание 1. Установка и исследование Prometheus**

1. Создать namespace monitoring.
2. Установить Prometheus с помощью Helm.
3. Проверить состояние.
4. Открыть Web-интерфейс Prometheus.
5. Выполнить запрос `up`.

Результат:

- команды установки;
- вывод `kubectl get pods -n monitoring`;
- скриншот Web-интерфейса;
- объяснение значения `up = 1`.

Задание 2. Исследование метрик

Найти в Prometheus метрики (2-3), связанные с:

- CPU;
- памятью;
- HTTP-запросами;
- ошибками;
- состоянием Pod'ов.

Для каждой метрики определить: название, что измеряет, единицу, тип (counter/gauge/histogram).

Результат: таблица:

Метрика	Что измеряет	Тип	Единица

Задание 3. Исследование приложения под нагрузкой

Запустить нагрузочный тест из ЛР1. Провести серию экспериментов: 10, 50, 200 VU.

Для каждого значения зафиксировать: RPS, p95, p99, errors, CPU, Memory.

Результат:

– таблица;

VU	RPS	p95 (ms)	p99 (ms)	Errors (%)	CPU (%)	Memory (MB)
10						
50						
200						

– использованные запросы PromQL;

- сравнение p95 из k6 и p95 из Prometheus для одного уровня нагрузки (например, 100 VU): привести оба значения, посчитать разницу в мс и %, объяснить, если это возможно причину расхождения
- краткий анализ результатов.

Задание 4. Поиск причины деградации

Используя результаты заданий 2 и 3, определить момент, в котором производительность системы начинает ухудшаться.

1. Найти нагрузку, при которой начинает расти p95.
2. Определить, как изменяется CPU.
3. Определить, как изменяется Memory.
4. Проверить изменение RPS.
5. Проверить количество ошибок.
6. Сформулировать гипотезу о возможной причине деградации.

Контрольные вопросы

1. Что такое мониторинг и зачем он нужен в высоконагруженных системах? Чем мониторинг отличается от нагрузочного тестирования?
2. Какую информацию даёт k6, а какую — Prometheus? Почему их показатели RPS и p95 могут не совпадать?
3. Что такое Prometheus и в чём заключается Pull-модель?
4. Что такое метрика, временной ряд и labels? Зачем нужны labels?
5. Чем counter отличается от gauge? Приведите примеры. Почему `rate()` нельзя применять к gauge?
6. Почему Histogram часто используют для анализа latency в Prometheus? Чем Histogram отличается от Summary? Как получить p95 через PromQL?
7. Что такое PromQL? Как определить количество HTTP 5xx-ошибок и их долю с его помощью?
8. Что такое bottleneck? Почему одна метрика (например, CPU = 90%) сама по себе не доказывает наличие bottleneck?
9. Что такое warm-up и saturation point? Почему первые секунды теста стоит игнорировать?
10. Чем гипотеза о bottleneck отличается от доказанного результата измерения? Как правильно сформулировать гипотезу по результатам эксперимента?